

[El títol del TFG]

[El subtítol del tfg: Opcional]

TREBALL DE FI DE GRAU DE
Aniol Petit Cabarrocas

Jordi Taixés i Ventosa

Mathematical Engineering in Data Science

Curs 2025 - 2026



Universitat
Pompeu Fabra
Barcelona

Escola
d'Enginyeria

Do or do not, there is no try - Master Yoda

Agraïments

dsds

Abstract

[Insert Abstract here]

Abstract (Catalan)

[Insert Abstract here]

Abstract (Spanish)

[Insert Abstract here]

Preface

Contents

List of Figures	xiii
List of Tables	xv
1 INTRODUCTION	1
1.1 Context and Motivation	1
1.2 Problem Statement	1
1.3 Objectives and Scope	1
2 THEORETICAL PRELIMINARIES	3
2.1 Graph Theory Foundations	3
2.1.1 Time-Dependent vs. Time-Expanded Networks	3
2.1.2 Activity-on-Edge (AoE) vs. Activity-on-Node (AoN)	4
2.2 Search and Optimization Algorithms	4
2.2.1 Dynamic Programming and Label Setting	4
2.2.2 Depth-First Search (DFS) and Branch-and-Bound	4
2.3 Distributed Computing	4
2.4 Data Acquisition Concepts	5
3 DATA ACQUISITION AND PROCESSING PIPELINE	7
3.1 Sourcing Transit Data	7
3.1.1 Flight APIs	7
3.1.2 Train Data	7
3.2 Data Cleaning and Normalization	7
3.3 Spatial Mapping and Filtering	7
3.4 Transfer Durations	7
4 ALGORITHMIC APPROACH I: LABEL SETTING AND BOUNDING FOUNDATIONS	9
4.1 The Time-Expanded Transportation Graph	9
4.2 Algorithm Evolution: From Baseline to Bounding	9
4.2.1 Baseline Label Setting and Pareto Frontiers	9
4.2.2 Upper Bound (UB) Pruning	9
4.2.3 Minimum Travel Time (MTT) Heuristic	9
4.3 The Dimensionality Collapse	9
5 ALGORITHMIC APPROACH II: GRAPH-BASED DFS EXPLORATION	11
5.1 The Activity-on-Node (AoN) Transformation	11
5.2 Baseline DFS and the Reduced Graph	11
5.3 Scaling to the Full Graph (Distributed DFS)	11
5.4 Optimizing the Search Space	11
5.4.1 State Dominance and the Pruning Bug	11
5.4.2 Heuristic Pre-Sort and RQI	11
	xi

6	RESULTS AND CONCLUSIONS	13
6.1	Dataset Generation	13
6.2	Analysis of Elite Routes	13
6.3	Conclusions	13
6.4	Future Work	13
A	SUPPORTING MATERIAL	15

List of Figures

List of Tables

1

Introduction

1.1 Context and Motivation

1.2 Problem Statement

1.3 Objectives and Scope

2

Theoretical Preliminaries

2.1 Graph Theory Foundations

To computationally solve a routing problem across the European transit network, the physical infrastructure must be translated into a mathematical structure known as a graph. A graph $G = (V, E)$ is defined by a set of vertices V (or nodes) and a set of edges E that connect pairs of vertices. In the context of transportation routing, a standard graph is inherently directed and weighted.

Directed edges. Transit connections are unilateral. An edge $e = (u, v)$ indicates a valid route from node u to node v . The existence of $e = (u, v)$ does not imply the existence of a return edge $e' = (v, u)$ at the same time or cost.

Weighted edges. Each edge carries a cost, typically represented by a weight function $w(u, v)$. In our context, this weight represents temporal costs, such as the travel duration of a flight or the waiting time of a layover.

A path in this graph is a sequence of vertices where each adjacent pair is connected by a valid edge. The goal of a routing algorithm is to find a specific path that maximizes or minimizes a certain objective function (e.g., minimizing total weight or maximizing the number of unique vertices visited).

2.1.1 Time-Dependent vs. Time-Expanded Networks

A traditional static graph is insufficient for modeling public transit because connections do not exist continuously; they are strictly bound to specific departure and arrival schedules. To model this dimension of time, routing problems typically rely on one of two paradigms:

Time-Dependent Networks. The physical topology of the graph remains static (e.g., one vertex per station), but the weight of an edge $w(u, v, t)$ is a function of the departure time t . This keeps the network compact, but complicates the routing algorithms, as the cost of traversal fluctuates dynamically.

Time-Expanded Networks. Time is explicitly discretized and baked directly into the topology of the graph. A single physical station is duplicated into multiple vertices, each representing that station at a specific point in time (e.g., Station A at 10:00, Station A at 10:15). This dramatically increases the number of nodes and edges, but allows the use of standard, static routing algorithms, since every edge now possesses a fixed, immutable weight.

Because our problem operates within a strict 24-hour window using discrete, fixed train and flight schedules, this thesis relies on time-expanded models.

2.1.2 Activity-on-Edge (AoE) vs. Activity-on-Node (AoN)

Within time-expanded models, the data can be structured in two distinct ways:

Activity-on-Edge (AoE). The traditional approach where vertices represent physical locations at specific times, and edges represent the actual transport activities (a train moving between locations) or waiting periods at a station.

Activity-on-Node (AoN). An inverted approach where the vertices themselves represent the specific transport activities (a flight with fixed departure and arrival times), and the edges represent the valid layovers or transfer opportunities connecting them.

Understanding the distinction between AoE and AoN is critical. Shifting between these two paradigms radically alters the branching factor and traversability of the search space, a structural pivot that will be central to the algorithmic evolution presented in this thesis.

2.2 Search and Optimization Algorithms

When graph networks become too large for brute-force enumeration, exact and heuristic optimization algorithms are required to traverse the search space efficiently.

2.2.1 Dynamic Programming and Label Setting

Dynamic Programming (DP) is a mathematical optimization method that solves complex problems by breaking them down into simpler, overlapping subproblems. In the context of graph routing, DP is often implemented via Label Setting algorithms. As the algorithm traverses the graph, it assigns “labels” to each node, where a label represents the accumulated cost or state of a specific path reaching that node. In multi-objective optimization problems—where multiple conflicting criteria must be optimized simultaneously (e.g., minimizing time while maximizing profit)—a single optimal path cannot always be determined dynamically. Instead, the algorithm must maintain a Pareto Frontier at each node. The frontier is a set containing all non-dominated labels. A label L_1 mathematically “dominates” L_2 if it is equal or superior in all evaluated objectives, and strictly superior in at least one. Dominated labels are permanently discarded, while the non-dominated set is kept to spawn future branches.

2.2.2 Depth-First Search (DFS) and Branch-and-Bound

Depth-First Search (DFS) is a fundamental graph traversal algorithm that explores as far down a single branch of a tree as possible before backtracking. Because DFS only requires storing the nodes along the current active path, its memory complexity grows linearly with the depth of the search tree rather than exponentially with its breadth, making it highly memory-efficient.

To optimize DFS for finding absolute maximums or minimums, it is often paired with Branch-and-Bound (BnB), a paradigm for discrete and combinatorial optimization.

Branching. The algorithm systematically divides the search space by exploring subsequent child nodes.

Bounding. At each node, a bounding function calculates an optimistic mathematical estimate (an upper or lower bound) of the best possible solution that could be found by continuing down that branch.

If the calculated bound of a branch is worse than the best solution already found elsewhere in the search (the incumbent solution), the algorithm instantly abandons the branch. This technique, known as pruning, eliminates vast sections of the search space without requiring explicit evaluation.

2.3 Distributed Computing

When a computational problem’s scale exceeds the memory or processing capacity of a single machine, distributed computing methodologies are employed to divide the task across a High-Performance Computing (HPC) cluster. An HPC cluster is a network of independent computers (nodes) that function as a

single system. Access to these resources is governed by a workload manager or job scheduler, such as SLURM (Simple Linux Utility for Resource Management). The scheduler allocates specific hardware constraints, such as CPU cores and maximum RAM, and queues tasks to ensure that execution does not exceed physical memory limits, which would otherwise trigger immediate task termination.

To efficiently search massive combinatorial state spaces, algorithms frequently leverage data-level parallelism. In this paradigm, the exact same algorithm is executed concurrently across non-overlapping subsets of the initial data. In HPC environments, this is seamlessly orchestrated using job arrays. A job array takes a single execution script and duplicates it into dozens or hundreds of independent tasks, assigning each a unique environmental index. The algorithm uses this index to deterministically identify and process its specific slice of the search space, achieving massive throughput without the overhead of constant inter-node network communication.

Within those individual tasks, intra-node multiprocessing is often utilized to fully saturate the assigned CPU cores. However, launching multiple concurrent processes can lead to severe memory duplication, which is fatal for memory-intensive graph traversals. To mitigate this, concurrent architectures employ Inter-Process Communication (IPC) and shared memory. By utilizing thread-safe locks and shared variables, multiple worker processes can safely read global data structures and update shared states, such as a global incumbent bound, without triggering race conditions or redundant memory allocation.

2.4 Data Acquisition Concepts

To populate large-scale routing networks, real-world scheduling data must be extracted programmatically from remote servers. The standard mechanism for this is a REST API (Representational State Transfer Application Programming Interface). APIs provide structured endpoints where a client can send HTTP requests to retrieve specific, filtered datasets. In modern web architectures, this data is almost universally exchanged in JSON (JavaScript Object Notation), a lightweight, key-value serialization format. JSON allows nested, hierarchical data to be seamlessly transmitted over networks and deserialized directly into programmatic data structures, such as dictionaries and arrays, for immediate computational use.

However, when public APIs are unavailable or heavily restricted, data must be gathered via web scraping. Web scraping involves automating HTTP requests to target servers and parsing the resulting HTML, a markup language designed for human-readable browsers rather than machine consumption. This requires algorithmic extraction of target variables from unstructured or semi-structured Document Object Model (DOM) trees. Both API querying and web scraping are strictly governed by server-side rate limits (throttling). Consequently, data acquisition pipelines must implement pagination logic, exponential backoff strategies, and fault-tolerant parsing to ensure reliable, large-scale dataset generation without triggering server bans or connection timeouts.

3

Data Acquisition and Processing Pipeline

3.1 Sourcing Transit Data

3.1.1 Flight APIs

3.1.2 Train Data

3.2 Data Cleaning and Normalization

3.3 Spatial Mapping and Filtering

3.4 Transfer Durations

4

Algorithmic Approach I: Label Setting and Bounding Foundations

4.1 The Time-Expanded Transportation Graph

4.2 Algorithm Evolution: From Baseline to Bounding

4.2.1 Baseline Label Setting and Pareto Frontiers

4.2.2 Upper Bound (UB) Pruning

4.2.3 Minimum Travel Time (MTT) Heuristic

4.3 The Dimensionality Collapse

5

Algorithmic Approach II: Graph-Based DFS Exploration

5.1 The Activity-on-Node (AoN) Transformation

5.2 Baseline DFS and the Reduced Graph

5.3 Scaling to the Full Graph (Distributed DFS)

5.4 Optimizing the Search Space

5.4.1 State Dominance and the Pruning Bug

5.4.2 Heuristic Pre-Sort and RQI

6

Results and Conclusions

6.1 Dataset Generation

6.2 Analysis of Elite Routes

6.3 Conclusions

6.4 Future Work



Supporting Material

[Additional tables and data will go here]